



17

Kick-Starting React Native Projects

In this chapter, you'll get up and running with React Native. Thankfully, much of the boilerplate code involved in the creation of a new project is handled for you by the command-line tools. We will look at the different CLI tools for React Native apps and create our first simple app, which you will be able to upload and start right on your device.

In this chapter, we'll cover the following topics:

- Exploring React Native CLI tools
- Installing and using the Expo command-line tool
- Viewing your app on your phone
- Viewing your app on Expo Snack

Technical requirements

You can find the code files of this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter17>.

Exploring React Native CLI tools

To simplify and speed up the development process, we use special command-line tools that install blank projects with application templates, dependencies, and other tools for starting, building, and testing. There are two major CLI approaches we can apply:

- The React Native CLI
- The Expo CLI

The **React Native CLI** is a tool created by Meta. The project is based on the original CLI tool and has three parts: native iOS and Android projects and a React Native JavaScript app. To get started, you will need either **Xcode** or **Android Studio**. One of the main advantages of the React Native CLI is its flexibility. You can connect any library with a Native module or directly write code to the native parts. However, all of this requires at least a basic understanding of mobile development.

The **Expo CLI** is just one part of the big ecosystem for developing React Native apps. **Expo** is a framework and a platform for universal React applications. Built around React Native and na-

tive platforms, it allows you to build, deploy, test, and rapidly iterate on iOS, Android, and web apps from a single JavaScript/TypeScript code base.

The Expo framework provides the following:

- The **Expo CLI**: A command-line tool that can create blank projects, then run, build, and update them.
- **Expo Go**: An Android and iOS app for running your projects directly on your device (without having to compile and sign native apps) and sharing them with your entire team.
- **Expo Snack**: The online playground that allows you to develop React Native apps in the browser.
- **Expo Application Services (EAS)**: A set of deeply integrated cloud services for Expo and React Native applications. Apps can be compiled, signed, and uploaded to stores using EAS in the cloud.

Expo comes with a huge number of ready-to-use features. Previously, it imposed limitations on projects, as it did not support custom native modules. However, this limitation no longer exists. Expo now supports adding custom native code and customizing that native code (Android/Xcode projects) through Expo development builds. To use any custom native code, you can create a development build and config plugins.

Since Expo is useful for new developers without mobile development skills, we will use it to set up our first React Native project.

Installing and using the Expo command-line tool

The **Expo command-line tool** handles the creation of all of the scaffolding that your project needs to run a basic React Native application. Additionally, Expo has a couple of other tools that make running our app during development nice and straightforward. But first, we need to set up the environment and project:

1. Before we can use Expo, we need to install **Node.js**, **Git**, and **Watchman**. Watchman is a tool for watching files in our project to trigger actions like rebuilds when they change. All of the required tools and details can be found here:
<https://docs.expo.dev/get-started/installation/#requirements>.
2. Once this installation is complete, we can start a new project by running the command:

```
npx create-expo-app --template
```

3. Next, the CLI will ask you questions about your future project. You should see something like this in your terminal:

```
? Choose a template: ' - Use arrow-keys. Return to submit.  
  Blank  
  > Blank (TypeScript) - blank app with TypeScript enabled  
    Navigation (TypeScript)  
    Blank (Bare)
```

We'll choose the `Blank (TypeScript)` option.

4. Next, the process will ask you about a project name:

```
? What is your app named? ' my-project
```

Let's call it `my-project`.

5. After installing all the dependencies, Expo will finish creating your project for you:

```
✓ Your project is ready!
```

Now that we have created a blank React Native project, you'll learn how to launch the Expo development server on your computer and view the app on one of your devices.

Viewing your app on your phone

In order to view your React Native project on your device during development, we need to start the Expo development server:

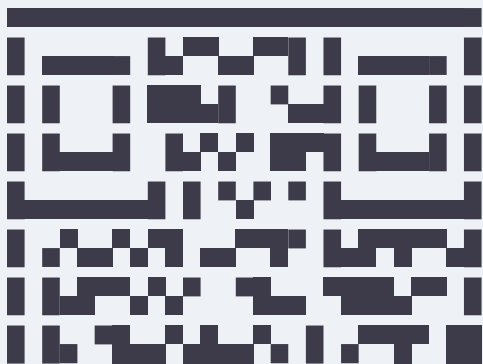
1. In the command-line terminal, make sure that you're in the project directory:

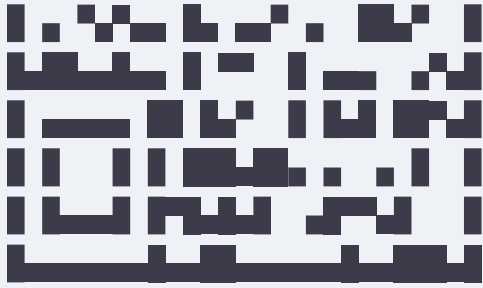
```
cd path/to/my-project
```

2. Once you're in `my-project`, you can run the following command to start the development server:

```
npm start
```

3. This will show you some information about the developer server in the terminal:





```
' Metro waiting on exp://192.168.1.15:8081
' Scan the QR code above with Expo Go (Android) or the Camera app (iOS)
' Using Expo Go
' Press s | switch to development build
' Press a | open Android
' Press i | open iOS simulator
' Press w | open web
' Press j | open debugger
' Press r | reload app
' Press m | toggle menu
' Press o | open project code in your editor
' Press ? | show all commands
```

4. In order to view the app on our devices, we need to install the **Expo Go** app. You can find it in the Play Store on Android devices or in the App Store on iOS devices. Once you have Expo installed, you scan the QR code using the native camera on your device:

17:15



Projects

TOOLS

Get started with Expo

Run projects from expo-cli or Snack.

RECENTLY IN DEVELOPMENT

HELP



my-project on Mikhails-MBP
exp://192.168.1.233:19000

RECENTLY OPENED

CLEAR

You haven't opened any projects recently.

Device ID: 44f3-1476
Client version: 2.23.2.1013698
Supported SDKs: 41, 42, 43, 44



Projects

Diagnostics

Profile



Figure 17.1: Expo Go app

If you log in to Expo Go and the Expo CLI, you will be able to run the app without the QR code. In *Figure 17.1*, you can see the opened development session for `my-project`; if you click on it, the app will run.

5. Once the QR code is scanned or your opened session on Expo Go is clicked, you'll notice new logs and a new connected device in the terminal:

```
iOS Bundling complete 205ms
```

6. Now you should see your app running:

17:29 ↗



Open up App.js to start working on your app!



Figure 17.2: Opened app in Expo Go

At this point, you're ready to start developing your app. In fact, you can repeat this same process if you have several physical devices that you want to work with at the same time. The best part of this Expo setup is that we get live reloading for free on our physical devices as we make code updates on our computers. Let's try this now to make sure that everything works as expected:

1. Let's open up the `App.ts` file inside the `my-project` folder. There, you'll see the `App` component:

```
export default function App() {  
  return (  
    <View style={styles.container}>  
      <Text>Open up App.tsx to start working on your app!</Text>  
      <StatusBar style="auto" />  
    </View>  
  );  
}
```

2. Now let's make a small style change to make the font bold:

```
export default function App() {  
  return (  
    <View style={styles.container}>  
      <Text style={styles.text}>  
        Open up App.tsx to start working on your app!  
      </Text>  
      <StatusBar style="auto" />  
    </View>  
  );  
}  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    backgroundColor: "#fff",  
    alignItems: "center",  
    justifyContent: "center",  
  },  
  text: { fontWeight: "bold" },  
});
```

3. We've added a new style called `text` and applied it to the `Text` component. If you save the file and return to your device, you'll immediately see the change applied:

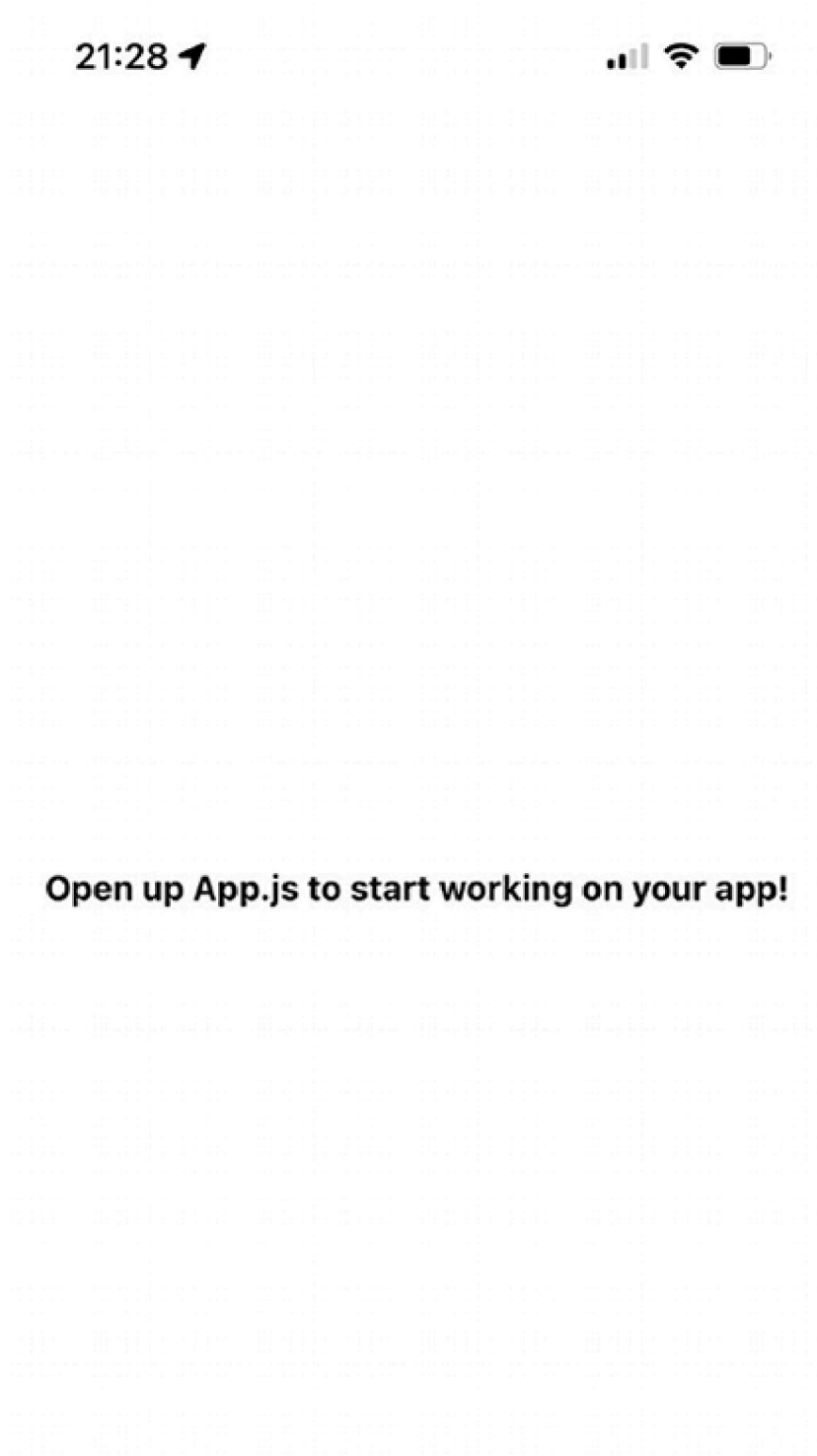




Figure 17.3: App with updates to style of text

Now that you're able to run your apps locally on your physical devices, it's time to look at running your React Native apps on a variety of virtual device emulators using the Expo Snack service.

Viewing your app on Expo Snack

The **Snack** service provided by Expo is a playground for your React Native code. It lets you organize your React Native project files just like you would locally on your computer. If you end up putting something together that is worth building on, you can export your Snack. You can also create an Expo account and save your Snacks to keep working on them or to share them with others. You can find Expo Snack with this link: <https://snack.expo.dev/>.

We can create a React Native app in Expo Snack from scratch, and it will be stored in an Expo account, or we can import existing projects from a Git repository. The nice thing about importing a repository is that when you push changes to Git, your

Snack will also be updated. The Git URL for the example that we've worked on in this chapter looks like this:

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter17/my-project>.

We can click on the **Import git repository** button in the Snack project menu and paste in this URL:

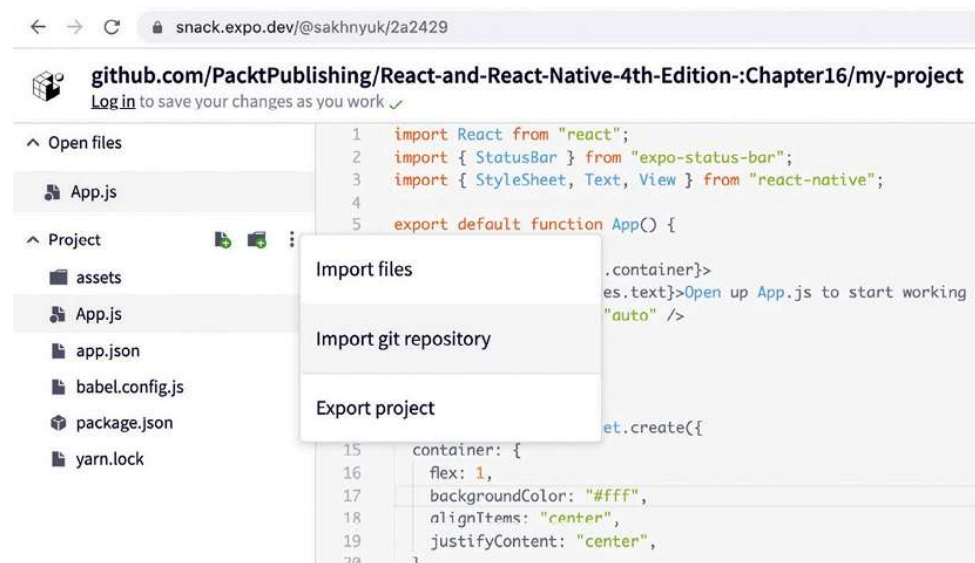


Figure 17.4: Importing a Git repository to Expo Snack

Once the repository is imported and the Snack is saved, you'll get an updated Snack URL that reflects the Git repository location. For example, the Snack URL from this chapter looks like this: <https://snack.expo.dev/@sakhnyuk/2a2429> .

If you open this URL, the Snack interface will load and you can make changes to the code to test things out before running them. The main advantage of Snack is the ability to easily run it on virtualized devices. The controls to run your app on a virtual device can be found on the right side of the UI and look like this:

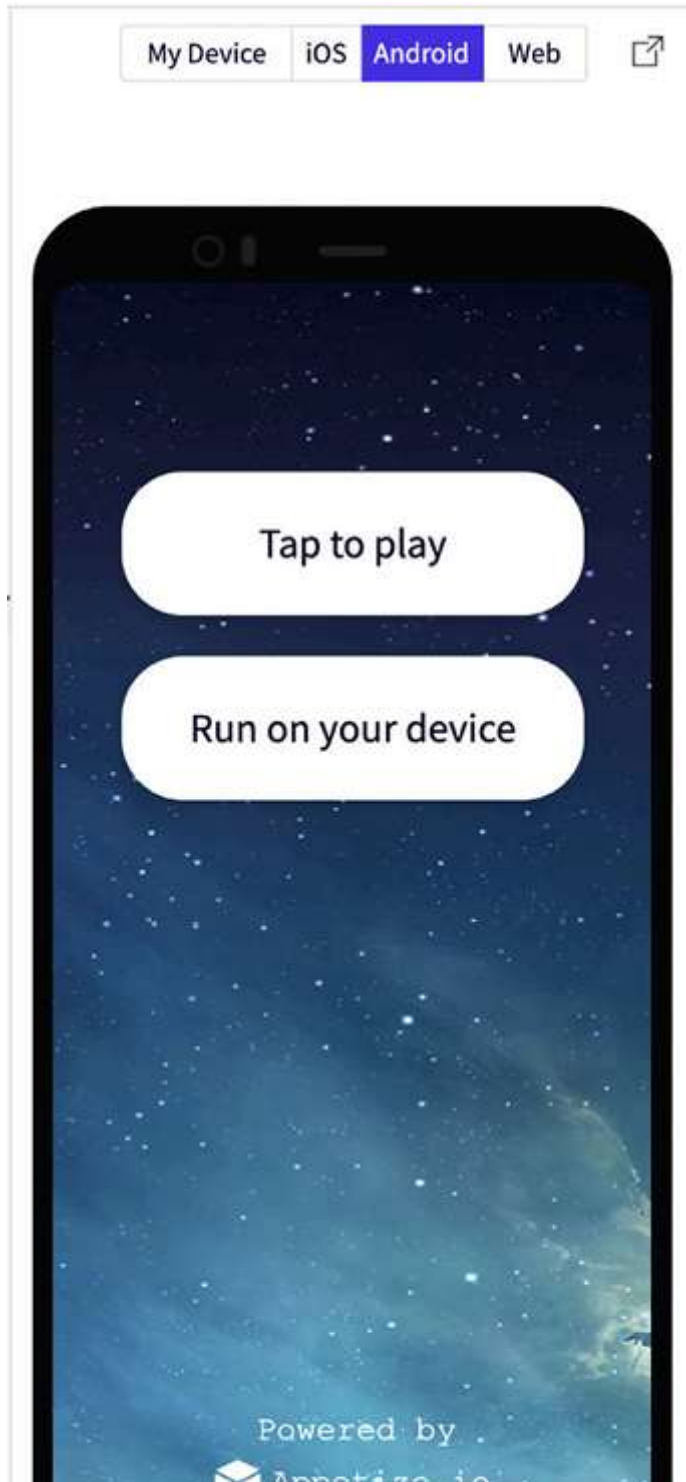




Figure 17.5: Expo Snack emulator

The top control above the image of the phone controls which device type to emulate: **Android**, **iOS**, or **Web**. The **Tap to play** button will launch the selected virtual device. The **Run on your device** button allows you to run the app in Expo Go using the QR code approach.

Here's what our app looks like on a virtual iOS device:

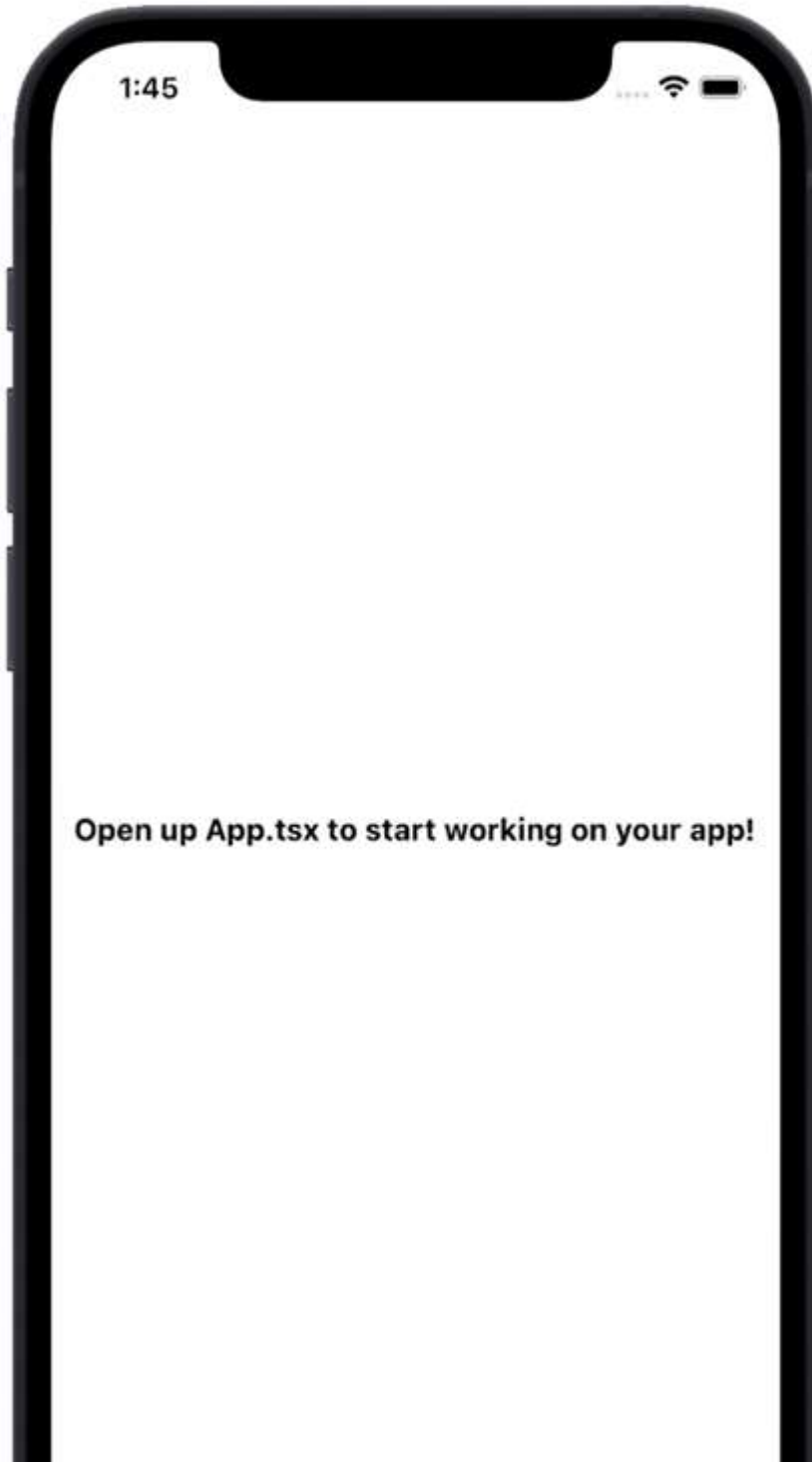




Figure 17.6: Expo Snack iOS emulator

And here's what our app looks like on a virtual Android device:



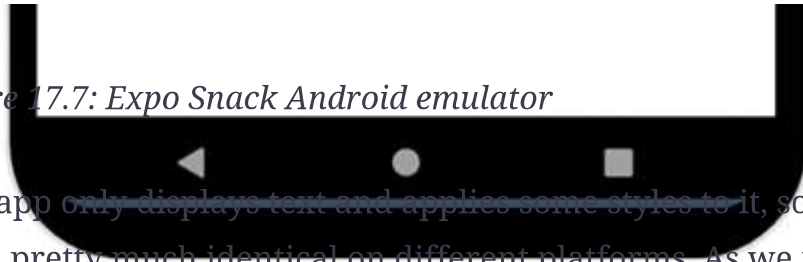


Figure 17.7: Expo Snack Android emulator

This app ~~only displays text and applies some styles to it~~, so it looks pretty much identical on different platforms. As we make our way through the React Native chapters in this book, you'll see how useful a tool such as Snack is for making comparisons between the two platforms to understand the differences between them.

Summary

In this chapter, you learned how to kick-start a React Native project using the Expo command-line tool. First, you learned how to install the Expo tool. Then, you learned how to initialize a new React Native project. Next, you started the Expo development server and learned about the various parts of the development server UI.

In particular, you learned how to connect the development server with the Expo app on any device that you want to test your app on. Expo also has the Snack service, which lets us experiment with snippets of code or entire Git repositories. You learned how to import a repository and run it on virtual iOS and Android devices.

In the next chapter, we'll look at how to build responsive layouts in our React Native apps.